Un guide gastronomique

Compétences développées dans le module

- Ecrire des requêtes d'interrogation à une table
  - Projections simples
  - Restrictions
- Ecrire des requêtes en affichant des résultats parlants pour l'utilisateur : renommage des colonnes de la table de résultat, tri des résultats.
- Ecrire des requêtes d'interrogation complexes à plusieurs tables
  - Jointures
- Faire des calculs sur une base de données
  - Avec des fonctions d'agrégation sans partition
  - Avec partition
- Ecrire des requêtes reposant sur des sous-requêtes
  - IN et NOT IN

Le syndicat d'initiative d'une grande ville a décidé de constituer un guide gastronomique des restaurants de la cité.
Ce guide doit renseigner les résidents, habitants et touristes sur les différents plats, et tout particulièrement les spécialités, que l'on peut déguster en ville et sur les vins mis en vente dans les restaurants.

La base de données permet d'avoir :

- Les **restaurants** de la ville, enregistrés avec un code d'identification, un nom, une adresse, les jours d'ouverture, leurs horaires d'ouverture à midi et le soir, la liste des plats proposés à la carte et la liste des vins proposés.
- Les **plats** que l'on peut déguster en ville. Pour chaque plat, on doit connaître son nom, sa place dans le repas (entrée, dessert, ...), la liste des prix dans les restaurants qui le proposent à leur carte, le nom éventuel du vin idéal (d'après un grand gastronome) et une couleur de vin recommandée permettant de choisir un vin adapté dans le cas où l'on ne peut prendre le vin idéal. Cette couleur peut être différente de la couleur du vin idéal.
- Les **vins** que l'on peut trouver en ville avec, pour chacun d'eux, des informations sur leur nom, la date de récolte du raisin, la région d'origine, la couleur, et la liste des prix dans les restaurants qui le proposent.

Voici le schéma relationnel qui a été proposé pour cette base :

Restaurants (<u>code</u>, nom, ville)
Vins (<u>code</u>, cru, millesime, region, couleur)
Plats (<u>code</u>, nom, place, couleurVinRdee, #codeVinIdeal)
ServirVin (<u>#codeRestau, #codeVin</u>, prixVin)
ServirPlat (<u>#codeRestau, #codePlat</u>, prixPlat, specialite)
Ouverture (<u>#codeRestau</u>, <u>libJour</u>, <u>horaireOuverture</u>, horaireFermeture)

Ecrire les requêtes SQL qui permettent de répondre aux questions ci-dessous.

TD5 et TP5 : Partitions (Suite)

5) Partition

Il serait intéressant de pouvoir effectuer tous ces calculs et toutes ces interrogations sur des partitions d'une table.
Par exemple :

Par restaurant : Quel est le prix moyen des vins par restaurant ? ***(Exemple 1)***
Par couleur : Quel est le nombre de vins par couleur ? ***(Exemple 2)***

Le **regroupement des lignes en partitions** se fait sur une ou plusieurs colonnes d'une table. Ensuite, la (ou les) fonction(s) d'agrégation s'applique(ront) à chacun de ces groupes.

Règles importantes :

- La clause **GROUP BY** se place toujours après la dernière condition du **WHERE**.
- Tous les colonnes présentes dans la **projection (clause SELECT)** et qui ne font pas l'objet d'une fonction d'agrégation doivent être incluses dans la clause **GROUP BY** (et non l'inverse).

Exemple 1 : *prix moyen des vins par restaurant*

```
SELECT codeRestau, AVG(prixVin)
FROM ServirVin
GROUP BY codeRestau
```

Cette requête calcule la moyenne du prix des vins **pour chaque restau.**

Exemple 2 : *nombre de vins par couleur*

```
SELECT couleur, Count(code)
FROM Vins
GROUP BY couleur
```

Cette requête calcule le nombre de vins **pour chaque couleur de vin.**

Points de vigilance !

- Si l'on souhaite, par exemple, calculer des informations pour un même client tout en affichant son nom, il ne suffit pas de grouper uniquement par **nom du client**, car plusieurs clients peuvent avoir le même nom. Dans ce cas, il faudra inclure le **code du client** dans la clause GROUP BY. Cela garantit que le regroupement se fait bien par client unique.
- Un GROUP BY dans une requête **sans fonction d'agrégation** est **un GROUP BY inutile.**

Exemple 1 : *liste des restaurants (nom) par ville*

Bonne Pratique :

```
SELECT ville, nom
FROM RESTAURANTS
ORDER BY ville, nom
```

Ici, on n'a pas utilisé de **GROUP BY** car il n'était pas nécessaire d'effectuer un calcul sur une partition.
En revanche, il est **obligatoire** d'utiliser un **ORDER BY** pour répondre à la requête et afficher clairement, pour chaque ville, les restaurants associés.

Mauvaise Pratique :

```
SELECT ville, nom
FROM RESTAURANTS
GROUP BY ville, nom
```

Cette requête **fonctionne**, mais elle effectue une **partition inutile** : les données sont groupées sans qu'aucune fonction d'agrégation ne soit appliquée. Le SGBD effectue un traitement supplémentaire **sans valeur ajoutée.**
=> **Le GROUP BY ici est donc inutile.**

Q20. Combien de plats sont proposés dans chaque restaurant (nom) ?

```
SELECT R.nom AS nomRestau, COUNT(*) AS nbPlats
FROM ServirPlat S
JOIN Restaurants R ON S.codeRestau = R.code
GROUP BY R.code, R.nom;
```

Q21. Quel est le prix minimale et maximale pour chaque plat (nom), trié par ordre alphabétique des plats?

```
SELECT P.nom AS nomPlat, MIN(S.prixPlat) AS prixMin, MAX(S.prixPlat) AS prixMax
FROM Plats P
JOIN ServirPlat S ON P.code = S.codePlat
GROUP BY P.code , P.nom
ORDER BY P.nom;
```

6) FILTRAGE par la clause HAVING

On peut vouloir ne sélectionner que certains des regroupements effectués par le **GROUP BY**. Ce filtrage se fait grâce à la clause **HAVING**.
HAVING permet d'appliquer une restriction sur les colonnes créées suite au regroupement, notamment sur les colonnes issues des fonctions d'agrégation telles que **SUM, MAX, etc.**

- La clause HAVING se place toujours **après** la clause GROUP BY.
- *Syntaxe : **HAVING** FONCTION(colonne) OPERATEUR VALEUR*

```
SELECT  codeRestau, Count(codePlat)
FROM  ServirPlat
GROUP BY codeRestau
HAVING   COUNT(codePlat)> 10
```

Cette requête renvoie le code de chaque restaurant servant plus de 10 plats.

```
SELECT  codeRestau, AVG(prixVin)
FROM  ServirVin
GROUP BY codeRestau,
HAVING AVG(prixVin)> 20
```

Cette requête renvoie la liste des codes des restaurants dont le prix moyen des vins est supérieur à 20.

Points de vigilance !

- La **valeur** utilisée dans la clause **HAVING** peut être une **valeur en dur** (10, 50…) ou le résultat d'un **calcul** ou d'une **sous-requête.**

Q22. Le prix moyen des plats et leur nom pour les plats dont le plus petit prix affiché est inférieur à 50 euros ?

```
SELECT P.nom AS nomPlat, AVG(S.prixPlat) AS prixMoyen
FROM Plats P
JOIN ServirPlat S ON P.code = S.codePlat
GROUP BY P.code, P.nom
HAVING MIN(S.prixPlat) < 50;
```

Q23. . Afficher les restaurants (nom et ville) « grande cave », c'est-à-dire ceux qui proposent un nombre de vins supérieur à 8, triés dans l'ordre décroissant du nombre de vins proposés ?

```
SELECT R.nom, R.ville
FROM Restaurants R
JOIN ServirVin S ON R.code = S.codeRestau
GROUP BY R.code, R.nom, R.ville
HAVING COUNT(*) > 8
ORDER BY COUNT(*) DESC;
```

Un peu de remue-méninges…

Q24. Quels sont les restaurants qui servent autant de plats que le restaurant « Zokoak » ?

```
SELECT R.nom AS nomRestau, COUNT(*) AS nbPlats
FROM Restaurants R
JOIN ServirPlat S ON R.code = S.codeRestau
WHERE R.nom <> 'Zokoak'
GROUP BY R.code, R.nom
HAVING COUNT(*) = (
        SELECT COUNT(*)
        FROM Restaurants R2
        JOIN ServirPlat S2 ON R2.code = S2.codeRestau
        WHERE R2.nom = 'Zokoak'
);
```

Q25. Quels sont les restaurants qui servent des vins dont le prix moyen est supérieur au prix moyen de l'ensemble des vins ?

```
SELECT R.nom AS nomRestau
FROM Restaurants R
JOIN ServirVin S ON R.code = S.codeRestau
GROUP BY R.code, R.nom
HAVING AVG(S.prixVin) > (
        SELECT AVG(S2.prixVin)
        FROM ServirVin S2
);
```